

File Types in the Turbulence–Droplet DNS Code

1 Overview

This code simulates forced, statistically homogeneous, isotropic turbulence with a Fourier (pseudo-spectral) method, one-way coupled to particles that mimic cloud droplets. The velocity field is evolved in Fourier space; particles are represented either as a continuum number-density field advected on the “slow manifold” (small Stokes number, i.e. particles that closely follow the fluid) or as explicit tracked particles (finite Stokes number, referred to as “big” particles). The code runs in parallel across many processors using MPI, with the simulation domain sliced into 1-D slabs along x .

2 Main simulation scripts (`forced-dns-*.py`)

These are five independent, largely duplicated top-level scripts, not a shared library with switches. Each hardcodes its own grid size, time step, save path, and physics; each is a program you run directly with `mpirun`, not a module you import.

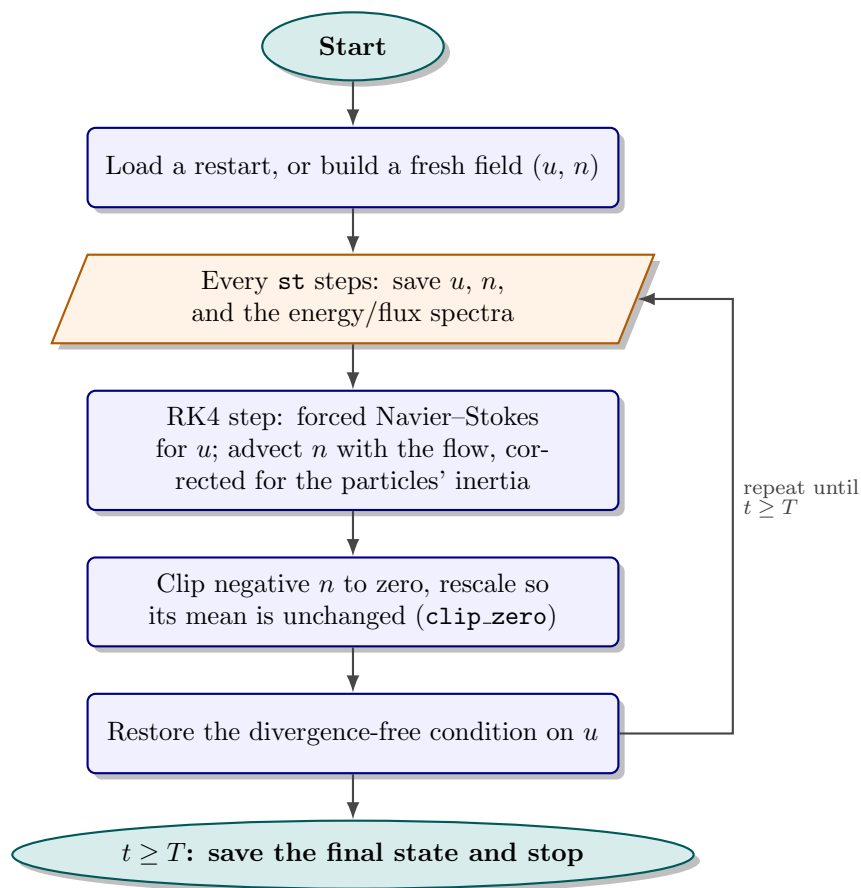
- `forced-dns-sm.py` — the baseline: forced Navier–Stokes turbulence plus the slow-manifold number-density field `n`, with no explicit particles. This is the file whose style the others imitate.
- `forced-dns-sm-clip-new.py` — identical to the baseline except that the crude step of clipping negative `n` to zero and rescaling is replaced by a root-finding procedure (`clip_zero`) that clips to an exact target mean.
- `forced-dns-log-sm.py` — evolves $\log(n)$ instead of `n` directly, which avoids needing to clip for positivity at all (a logarithm is automatically positive when exponentiated). Uses a different time step, save interval, and forcing amplitude, and its evolution equation carries different terms.
- `forced-dns-sm-big.py` — adds explicit tracked (“big”) particles for several Stokes numbers and initial conditions simultaneously, in addition to the slow-manifold field. This is the only variant that places new particles using the Q -criterion (a way of seeding particles in regions of strong flow rotation).
- `forced-dns-sm-big-rndm.py` — a big-particle variant with added random forcing on the particles themselves. It carries no density field at all.

Each script follows the same internal layout: parameter block and grid/Fourier-transform setup, a forcing routine, a clipping routine, the spectral right-hand side of the evolution equations, a fourth-order Runge–Kutta time-stepper, a checkpoint-saving routine, and a main time-stepping loop, followed by construction of the initial conditions and the call that runs the simulation.

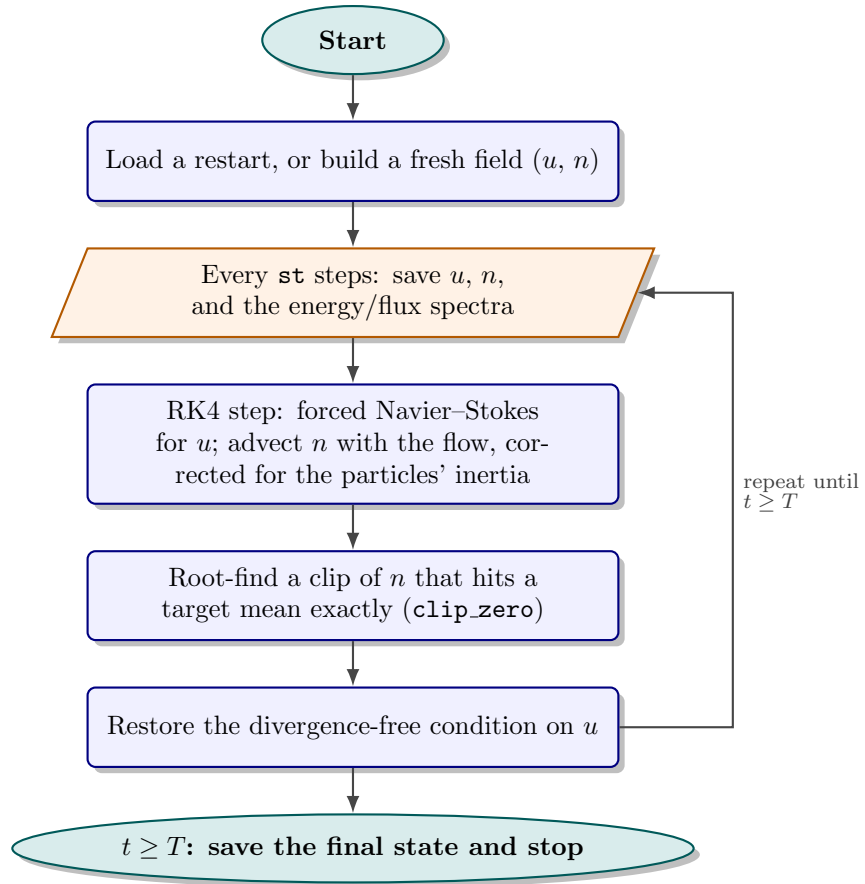
2.1 Control flow of each variant

All five scripts share the same outer skeleton: build or load the starting state, then repeatedly take a Runge–Kutta step, fix up the result, and save on a schedule, until the end time is reached. The flowcharts below highlight exactly where each variant departs from that skeleton.

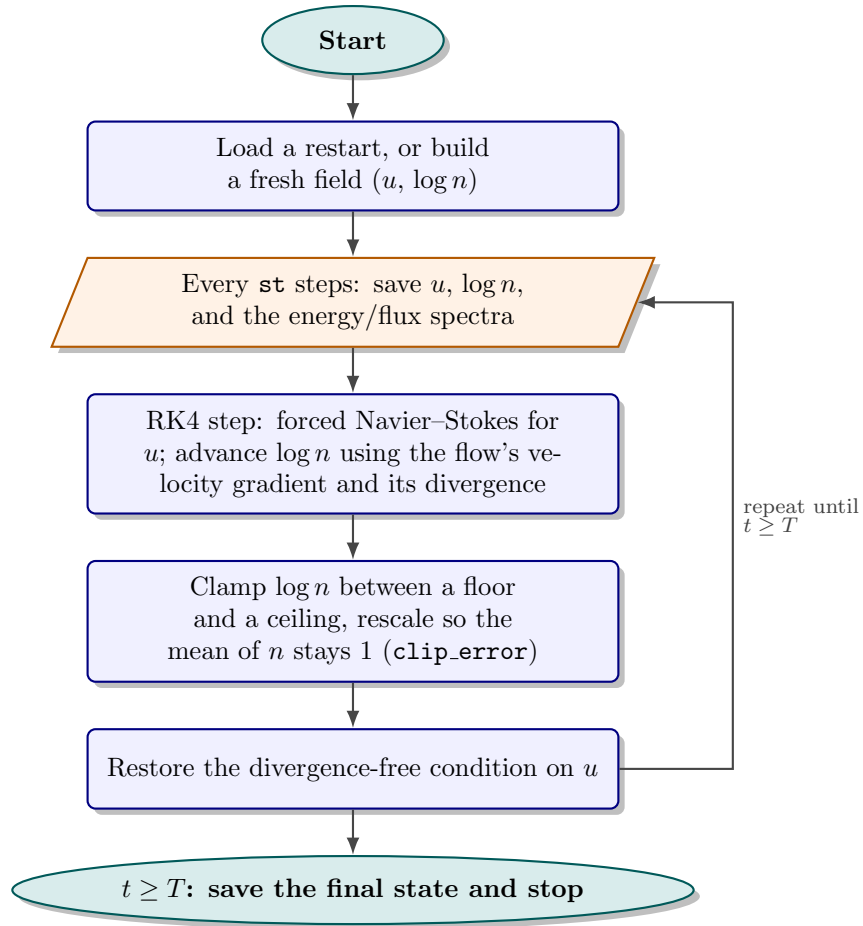
`forced-dns-sm.py`



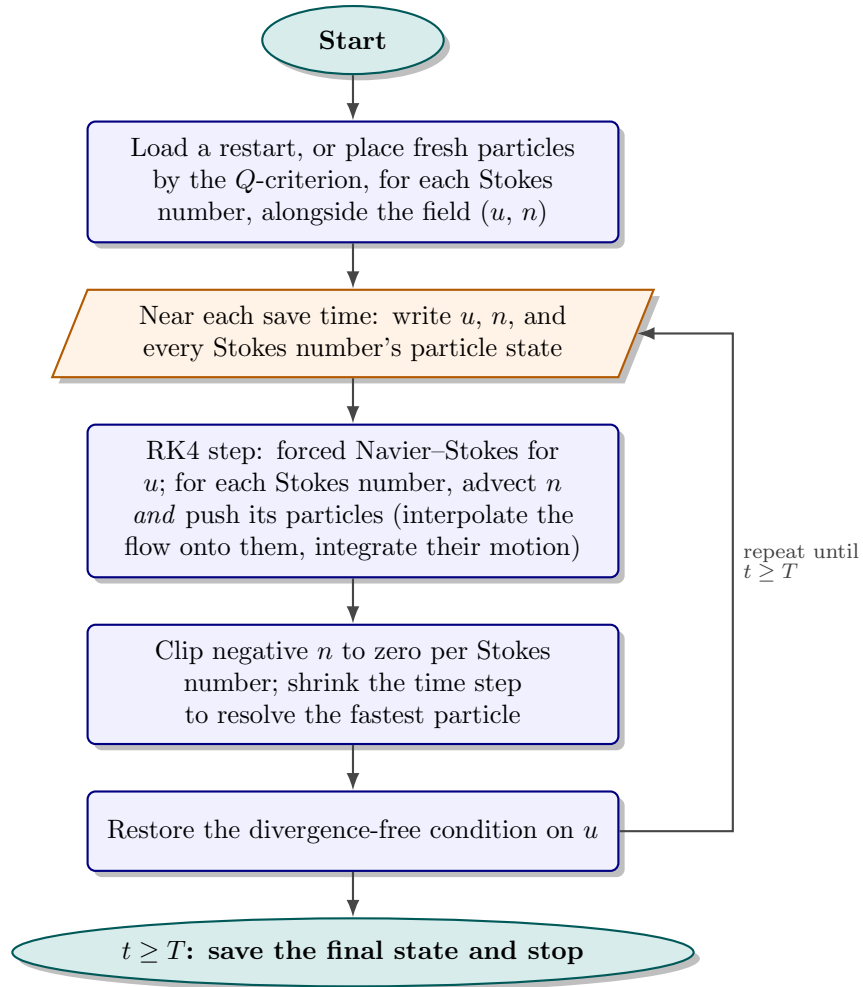
`forced-dns-sm-clip-new.py`



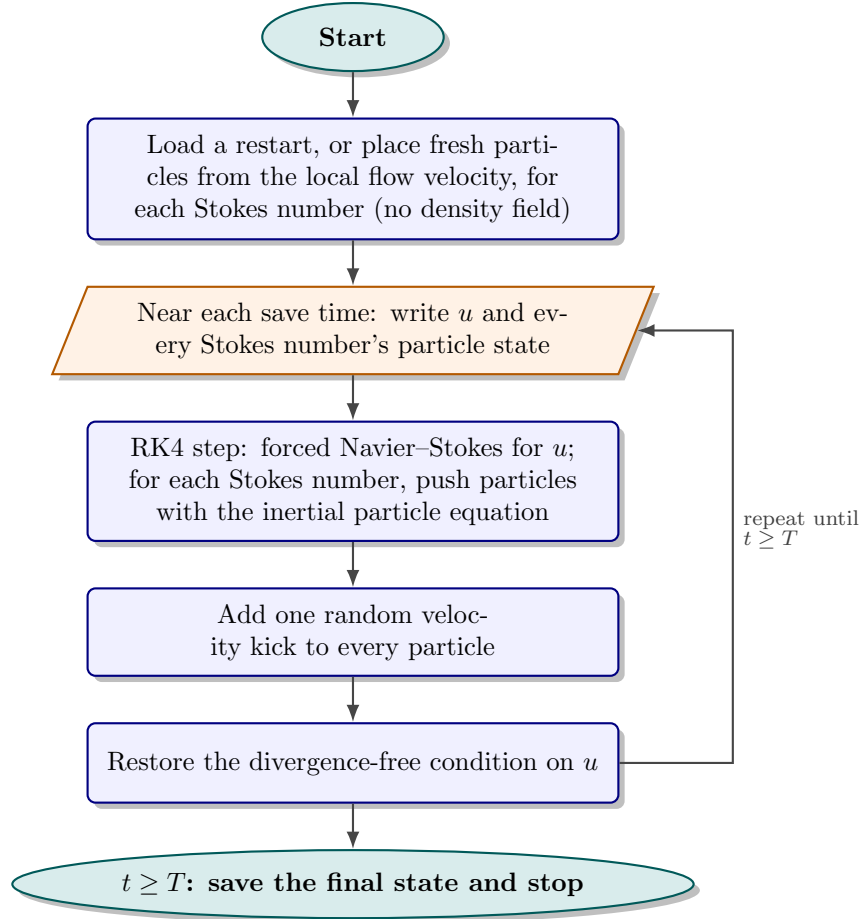
forced-dns-log-sm.py



forced-dns-sm-big.py



forced-dns-sm-big-rndm.py



3 Shared support modules

- `initial_conditions.py` — shared startup logic used by all five scripts. It either loads a previously saved state (a restart) or builds a fresh one, and is parameterized (via constructor arguments) to handle every difference between the five scripts' restart conventions: which checkpoint folder to use, how to clip the density, whether the field on disk is n or $\log n$, how particles are seeded from scratch, and so on.
- `particles.py` — the tracked-particle engine (class `MPI_particles`). Holds each particle's position, velocity, and mass; interpolates the fluid velocity onto particle positions and deposits particle quantities back onto the grid; exchanges particles between processors when they cross slab boundaries; and computes the particle equations of motion, including the random-forcing option. A few core interpolation routines are compiled just-in-time for speed and must stay compatible with that compiler's restrictions.
- `restart.py` — an earlier, now-superseded version of the restart logic. No script imports it, so it has no effect on any run; it is kept only as a leftover.

4 Analysis and plotting scripts

`plot.py`, `plot-correlation.py`, and the dated `plot-YYYYMMDD.py` files are standalone, notebook-style scripts (organized in cells) that read simulation output and produce figures. They are not imported by any other file and are treated as disposable: a new plotting need is normally addressed by adding a new dated file rather than editing an old one.

5 Data

`data_cosine/` holds a small local sample of simulation output, organized by forcing flag, grid resolution, and Reynolds number, then by save time. Each save-time folder contains Fourier-space velocity field pieces (one file per processor slab), energy- and flux-spectrum diagnostics, and, per Stokes number, either number-density files or tracked-particle state files. This sample data is excluded from version control and is much smaller than the full dataset, which lives on the cluster where the simulations actually run.

`comparing_spectra.hdf5` is a single self-describing binary data file (HDF5 format) at the repository root, used for comparing energy spectra.

6 Repository metadata

- `CLAUDE.md` — instructions and architectural notes for the Claude assistant working in this repository.
- `.gitignore` — lists paths excluded from version control (the sample data folder, compiled-code caches, and the ignore file itself).
- `.claude/` — assistant configuration, including the agent that reviews each response's changes for unnecessary complexity.